



多项式算法与生成函数

神秘多项式



目录 I

1 多项式算法

2 多项式插值

3 生成函数



目录

1 多项式算法

2 多项式插值

3 生成函数



问题介绍

我们要解决的问题是这样的，有两个长度（就是多项式的最高次幂）分别为 n, m 的多项式 $F(x), G(x)$ ，把它们乘到一起，得到新多项式 $H(x) = F(x)G(x)$ ，快速求出 H 的系数。



问题介绍

我们要解决的问题是这样的，有两个长度（就是多项式的最高次幂）分别为 n, m 的多项式 $F(x), G(x)$ ，把它们乘到一起，得到新多项式 $H(x) = F(x)G(x)$ ，快速求出 H 的系数。一般会有以下几种做法（认为 $O(n) = O(m)$ ）：



问题介绍

我们要解决的问题是这样的，有两个长度（就是多项式的最高次幂）分别为 n, m 的多项式 $F(x), G(x)$ ，把它们乘到一起，得到新多项式 $H(x) = F(x)G(x)$ ，快速求出 H 的系数。

一般会有以下几种做法（认为 $O(n) = O(m)$ ）：

- 直接暴力，时间复杂度是 $O(n^2)$ ，如果不是算法瓶颈，我们都会使用这个做法。



问题介绍

我们要解决的问题是这样的，有两个长度（就是多项式的最高次幂）分别为 n, m 的多项式 $F(x), G(x)$ ，把它们乘到一起，得到新多项式 $H(x) = F(x)G(x)$ ，快速求出 H 的系数。

一般会有以下几种做法（认为 $O(n) = O(m)$ ）：

- 直接暴力，时间复杂度是 $O(n^2)$ ，如果不是算法瓶颈，我们都会使用这个做法。
- karatsuba 算法，时间复杂度是 $O(3^{\log_2 n})$ ，也就是 $O(n^{1.585})$ ，其实常数很小，如果仅求少量的多项式乘法并且数据范围不大可以写这个。



问题介绍

我们要解决的问题是这样的，有两个长度（就是多项式的最高次幂）分别为 n, m 的多项式 $F(x), G(x)$ ，把它们乘到一起，得到新多项式 $H(x) = F(x)G(x)$ ，快速求出 H 的系数。

一般会有以下几种做法（认为 $O(n) = O(m)$ ）：

- 直接暴力，时间复杂度是 $O(n^2)$ ，如果不是算法瓶颈，我们都会使用这个做法。
- karatsuba 算法，时间复杂度是 $O(3^{\log_2 n})$ ，也就是 $O(n^{1.585})$ ，其实常数很小，如果仅求少量的多项式乘法并且数据范围不大可以写这个。
- FFT 算法，时间复杂度是 $O(n \log n)$ ，不过计算的过程中会使用复数与三角函数，运算为浮点运算且有误差。



问题介绍

我们要解决的问题是这样的，有两个长度（就是多项式的最高次幂）分别为 n, m 的多项式 $F(x), G(x)$ ，把它们乘到一起，得到新多项式 $H(x) = F(x)G(x)$ ，快速求出 H 的系数。

一般会有以下几种做法（认为 $O(n) = O(m)$ ）：

- 直接暴力，时间复杂度是 $O(n^2)$ ，如果不是算法瓶颈，我们都会使用这个做法。
- karatsuba 算法，时间复杂度是 $O(3^{\log_2 n})$ ，也就是 $O(n^{1.585})$ ，其实常数很小，如果仅求少量的多项式乘法并且数据范围不大可以写这个。
- FFT 算法，时间复杂度是 $O(n \log n)$ ，不过计算的过程中会使用复数与三角函数，运算为浮点运算且有误差。
- NTT 算法，时间复杂度同上，是 FFT 的改进算法，当模数有特殊性质时，比如 998244353，才可以使用。



问题介绍

我们要解决的问题是这样的，有两个长度（就是多项式的最高次幂）分别为 n, m 的多项式 $F(x), G(x)$ ，把它们乘到一起，得到新多项式 $H(x) = F(x)G(x)$ ，快速求出 H 的系数。

一般会有以下几种做法（认为 $O(n) = O(m)$ ）：

- 直接暴力，时间复杂度是 $O(n^2)$ ，如果不是算法瓶颈，我们都会使用这个做法。
- karatsuba 算法，时间复杂度是 $O(3^{\log_2 n})$ ，也就是 $O(n^{1.585})$ ，其实常数很小，如果仅求少量的多项式乘法并且数据范围不大可以写这个。
- FFT 算法，时间复杂度是 $O(n \log n)$ ，不过计算的过程中会使用复数与三角函数，运算为浮点运算且有误差。
- NTT 算法，时间复杂度同上，是 FFT 的改进算法，当模数有特殊性质时，比如 998244353，才可以使用。

我们会从第二个算法开始介绍。



karatsuba 算法

考虑分治求解问题。



karatsuba 算法

考虑分治求解问题。

令 n, m 相等且都是偶数（否则补充若干个为 0 的最高位）。



karatsuba 算法

考虑分治求解问题。

令 n, m 相等且都是偶数（否则补充若干个为 0 的最高位）。

不妨设 $F(x) = x^{n/2}F_1(x) + F_0(x)$ ，也就是把 $F(x)$ 分成了高于 $n/2$ 次幂的部分与低于 $n/2$ 的部分。

同理对 G 也做一个划分得到 $G_1(x), G_0(x)$ 。



karatsuba 算法

考虑分治求解问题。

令 n, m 相等且都是偶数（否则补充若干个为 0 的最高位）。

不妨设 $F(x) = x^{n/2} F_1(x) + F_0(x)$ ，也就是把 $F(x)$ 分成了高于 $n/2$ 次幂的部分与低于 $n/2$ 的部分。

同理对 G 也做一个划分得到 $G_1(x), G_0(x)$ 。

于是有 $H(x) = (x^{n/2} F_1 + F_0)(x^{n/2} G_1 + G_0) = x^n F_1 G_1 + x^{n/2}(F_1 G_0 + F_0 G_1) + F_0 G_0$ 。



karatsuba 算法

考虑分治求解问题。

令 n, m 相等且都是偶数（否则补充若干个为 0 的最高位）。

不妨设 $F(x) = x^{n/2} F_1(x) + F_0(x)$ ，也就是把 $F(x)$ 分成了高于 $n/2$ 次幂的部分与低于 $n/2$ 的部分。

同理对 G 也做一个划分得到 $G_1(x), G_0(x)$ 。

于是有 $H(x) = (x^{n/2} F_1 + F_0)(x^{n/2} G_1 + G_0) = x^n F_1 G_1 + x^{n/2}(F_1 G_0 + F_0 G_1) + F_0 G_0$ 。

乍一看直接暴力递归下去复杂度还是不变的，但是做一个小小的处理：

$F_1 G_0 + F_0 G_1 = (F_0 + F_1)(G_0 + G_1) - F_1 G_1 - F_0 G_0$ ，那么三项总共只用做三次规模为一半的子问题。



karatsuba 算法

考虑分治求解问题。

令 n, m 相等且都是偶数（否则补充若干个为 0 的最高位）。

不妨设 $F(x) = x^{n/2} F_1(x) + F_0(x)$ ，也就是把 $F(x)$ 分成了高于 $n/2$ 次幂的部分与低于 $n/2$ 的部分。

同理对 G 也做一个划分得到 $G_1(x), G_0(x)$ 。

于是有 $H(x) = (x^{n/2} F_1 + F_0)(x^{n/2} G_1 + G_0) = x^n F_1 G_1 + x^{n/2}(F_1 G_0 + F_0 G_1) + F_0 G_0$ 。

乍一看直接暴力递归下去复杂度还是不变的，但是做一个小小的处理：

$F_1 G_0 + F_0 G_1 = (F_0 + F_1)(G_0 + G_1) - F_1 G_1 - F_0 G_0$ ，那么三项总共只用做三次规模为一半的子问题。

所以时间复杂度为 $T(n) = 3T(n/2)$ ，也就是 $O(n^{1.585})$ 。



FFT 快速傅里叶变换算法

算法的思路其实很自然：



FFT 快速傅里叶变换算法

算法的思路其实很自然：

我们还记得一件事情，一个 n 次多项式，只需要 $n + 1$ 个它的点值就可以唯一确定他下来，因此我们考虑快速求出 $H(x)$ 的 $O(n + m)$ 个点值，再考虑快速插值回去。



FFT 快速傅里叶变换算法

算法的思路其实很自然：

我们还记得一件事情，一个 n 次多项式，只需要 $n + 1$ 个它的点值就可以唯一确定他下来，因此我们考虑快速求出 $H(x)$ 的 $O(n + m)$ 个点值，再考虑快速插值回去。

而 $H(x)$ 的点值 $H(x_0)$ ，其实就是 $F(x_0) \times G(x_0)$ 这两个数字乘积。因此只需要快速求出 F, G 的点值即可得到 H 的点值。



FFT 快速傅里叶变换算法

算法的思路其实很自然：

我们还记得一件事情，一个 n 次多项式，只需要 $n + 1$ 个它的点值就可以唯一确定他下来，因此我们考虑快速求出 $H(x)$ 的 $O(n + m)$ 个点值，再考虑快速插值回去。

而 $H(x)$ 的点值 $H(x_0)$ ，其实就是 $F(x_0) \times G(x_0)$ 这两个数字乘积。因此只需要快速求出 F, G 的点值即可得到 H 的点值。

我们不妨称 DFT 为使用快速傅里叶算法求得点值的过程，IDFT 为用 DFT 所求的点值快速求出多项式系数的过程。



FFT 快速傅里叶变换算法

考虑 DFT。



FFT 快速傅里叶变换算法

考虑 DFT。

我们如何快速求出来点值，第一步就是选取合适的点值。这里我们采用单位根上的点，即我们不妨找到第一个 $N = 2^k > n + m$ ，计算所有的 $F(\omega_N^i)$ 。显然这样足够获得至少 $m + n + 1$ 个点值，并且 2^k 作为单位根下标可以带来许多良好的性质，我们接下来就探讨这些性质。



FFT 快速傅里叶变换算法

考虑 DFT。

我们如何快速求出来点值，第一步就是选取合适的点值。这里我们采用单位根上的点，即我们不妨找到第一个 $N = 2^k > n + m$ ，计算所有的 $F(\omega_N^i)$ 。显然这样足够获得至少 $m + n + 1$ 个点值，并且 2^k 作为单位根下标可以带来许多良好的性质，我们接下来就探讨这些性质。首先我们知道 $\omega_N^{k+N/2} = -\omega_N^k$ ，以及 $\omega_N^k = \omega_{N/2}^{k/2}$ (k 是偶数)。



FFT 快速傅里叶变换算法

考虑 DFT。

我们如何快速求出来点值，第一步就是选取合适的点值。这里我们采用单位根上的点，即我们不妨找到第一个 $N = 2^k > n + m$ ，计算所有的 $F(\omega_N^i)$ 。显然这样足够获得至少 $m + n + 1$ 个点值，并且 2^k 作为单位根下标可以带来许多良好的性质，我们接下来就探讨这些性质。

首先我们知道 $\omega_N^{k+N/2} = -\omega_N^k$ ，以及 $\omega_N^k = \omega_{N/2}^{k/2}$ (k 是偶数)。

我们设 $F(x) = F_1(x^2) + xF_0(x^2)$ ，就是把系数奇偶分类一下，不难发现，这两个多项式的系数减半了。



FFT 快速傅里叶变换算法

考虑 DFT。

我们如何快速求出来点值，第一步就是选取合适的点值。这里我们采用单位根上的点，即我们不妨找到第一个 $N = 2^k > n + m$ ，计算所有的 $F(\omega_N^i)$ 。显然这样足够获得至少 $m + n + 1$ 个点值，并且 2^k 作为单位根下标可以带来许多良好的性质，我们接下来就探讨这些性质。

首先我们知道 $\omega_N^{k+N/2} = -\omega_N^k$ ，以及 $\omega_N^k = \omega_{N/2}^{k/2}$ (k 是偶数)。

我们设 $F(x) = F_1(x^2) + xF_0(x^2)$ ，就是把系数奇偶分类一下，不难发现，这两个多项式的系数减半了。如果我递归求出 $F_1(x)$ 和 $F_0(x)$ 在 $\omega_{N/2}^i$ 上的所有点值，再对每一项 $O(1)$ 合并求出 $F(x)$ 在 ω_N^i 上的点值，就做完了。



FFT 快速傅里叶变换算法

考虑 DFT。

我们如何快速求出来点值，第一步就是选取合适的点值。这里我们采用单位根上的点，即我们不妨找到第一个 $N = 2^k > n + m$ ，计算所有的 $F(\omega_N^i)$ 。显然这样足够获得至少 $m + n + 1$ 个点值，并且 2^k 作为单位根下标可以带来许多良好的性质，我们接下来就探讨这些性质。

首先我们知道 $\omega_N^{k+N/2} = -\omega_N^k$ ，以及 $\omega_N^k = \omega_{N/2}^{k/2}$ (k 是偶数)。

我们设 $F(x) = F_1(x^2) + xF_0(x^2)$ ，就是把系数奇偶分类一下，不难发现，这两个多项式的系数减半了。如果我递归求出 $F_1(x)$ 和 $F_0(x)$ 在 $\omega_{N/2}^i$ 上的所有点值，再对每一项 $O(1)$ 合并求出 $F(x)$ 在 ω_N^i 上的点值，就做完了。

事实上这可以做到，若 $i < N/2$ ， $F(\omega_N^i) = F_1(\omega_N^{2i}) + \omega_N^i F_0(\omega_N^{2i}) = F_1(\omega_{N/2}^i) + \omega_N^i F_0(\omega_{N/2}^i)$ 。



FFT 快速傅里叶变换算法

考虑 DFT。

我们如何快速求出来点值，第一步就是选取合适的点值。这里我们采用单位根上的点，即我们不妨找到第一个 $N = 2^k > n + m$ ，计算所有的 $F(\omega_N^i)$ 。显然这样足够获得至少 $m + n + 1$ 个点值，并且 2^k 作为单位根下标可以带来许多良好的性质，我们接下来就探讨这些性质。

首先我们知道 $\omega_N^{k+N/2} = -\omega_N^k$ ，以及 $\omega_N^k = \omega_{N/2}^{k/2}$ (k 是偶数)。

我们设 $F(x) = F_1(x^2) + xF_0(x^2)$ ，就是把系数奇偶分类一下，不难发现，这两个多项式的系数减半了。如果我递归求出 $F_1(x)$ 和 $F_0(x)$ 在 $\omega_{N/2}^i$ 上的所有点值，再对每一项 $O(1)$ 合并求出 $F(x)$ 在 ω_N^i 上的点值，就做完了。

事实上这可以做到，若 $i < N/2$ ， $F(\omega_N^i) = F_1(\omega_N^{2i}) + \omega_N^i F_0(\omega_N^{2i}) = F_1(\omega_{N/2}^i) + \omega_N^i F_0(\omega_{N/2}^i)$ 。

同理得到若 $i + n/2 \geq N/2$ ， $F(\omega_N^{i+n/2}) = F_1(\omega_{N/2}^i) - \omega_N^i F_0(\omega_{N/2}^i)$ 。



FFT 快速傅里叶变换算法

考虑 DFT。

我们如何快速求出来点值，第一步就是选取合适的点值。这里我们采用单位根上的点，即我们不妨找到第一个 $N = 2^k > n + m$ ，计算所有的 $F(\omega_N^i)$ 。显然这样足够获得至少 $m + n + 1$ 个点值，并且 2^k 作为单位根下标可以带来许多良好的性质，我们接下来就探讨这些性质。

首先我们知道 $\omega_N^{k+N/2} = -\omega_N^k$ ，以及 $\omega_N^k = \omega_{N/2}^{k/2}$ (k 是偶数)。

我们设 $F(x) = F_1(x^2) + xF_0(x^2)$ ，就是把系数奇偶分类一下，不难发现，这两个多项式的系数减半了。如果我递归求出 $F_1(x)$ 和 $F_0(x)$ 在 $\omega_{N/2}^i$ 上的所有点值，再对每一项 $O(1)$ 合并求出 $F(x)$ 在 ω_N^i 上的点值，就做完了。

事实上这可以做到，若 $i < N/2$ ， $F(\omega_N^i) = F_1(\omega_N^{2i}) + \omega_N^i F_0(\omega_N^{2i}) = F_1(\omega_{N/2}^i) + \omega_N^i F_0(\omega_{N/2}^i)$ 。

同理得到若 $i + n/2 \geq N/2$ ， $F(\omega_N^{i+n/2}) = F_1(\omega_{N/2}^i) - \omega_N^i F_0(\omega_{N/2}^i)$ 。

于是可以 $O(n \log n)$ 求解出点值。



FFT 快速傅里叶变换算法

考虑 IDFT。

FFT 快速傅里叶变换算法

考虑 IDFT。

设矩阵 $A_{n \times n}$, $A_{i,j} = \omega^{ij}$ 。假如 $F(x)$ 的系数构成的向量为 C , 那么 DFT 所求出来的就是 CA 的结果, 我想得到一个矩阵 B , 它可以做到 $CAB = C$, 所以 B 也就是 A 的逆。那么 $B = A^{-1}$, 只要求出来了 B , 那么对 $H(x)$ 的点值数组乘上 B 就可以得到答案了。



FFT 快速傅里叶变换算法

考虑 IDFT。

设矩阵 $A_{n \times n}$, $A_{i,j} = \omega^{ij}$ 。假如 $F(x)$ 的系数构成的向量为 C , 那么 DFT 所求出来的就是 CA 的结果, 我想得到一个矩阵 B , 它可以做到 $CAB = C$, 所以 B 也就是 A 的逆。那么 $B = A^{-1}$, 只要求出来了 B , 那么对 $H(x)$ 的点值数组乘上 B 就可以得到答案了。

结论就是 $B_{i,j} = \frac{\omega^{-ij}}{N}$ 。证明很简单, 直接把 AB 乘起来, 发现只有 $i = j$ 时结果为 1, 其余为 0。



FFT 快速傅里叶变换算法

考虑 IDFT。

设矩阵 $A_{n \times n}$, $A_{i,j} = \omega^{ij}$ 。假如 $F(x)$ 的系数构成的向量为 C , 那么 DFT 所求出来的就是 CA 的结果, 我想得到一个矩阵 B , 它可以做到 $CAB = C$, 所以 B 也就是 A 的逆。那么 $B = A^{-1}$, 只要求出来了 B , 那么对 $H(x)$ 的点值数组乘上 B 就可以得到答案了。

结论就是 $B_{i,j} = \frac{\omega^{-ij}}{N}$ 。证明很简单, 直接把 AB 乘起来, 发现只有 $i = j$ 时结果为 1, 其余为 0。

于是考虑快速计算 CB 的结果, 其实和 DFT 是一样的, 不妨把这个 $\frac{1}{n}$ 放到最后再除, 其他无非就是把单位根幂次换成了负的。



FFT 快速傅里叶变换算法

考虑 IDFT。

设矩阵 $A_{n \times n}$, $A_{i,j} = \omega^{ij}$ 。假如 $F(x)$ 的系数构成的向量为 C , 那么 DFT 所求出来的就是 CA 的结果, 我想得到一个矩阵 B , 它可以做到 $CAB = C$, 所以 B 也就是 A 的逆。那么 $B = A^{-1}$, 只要求出来了 B , 那么对 $H(x)$ 的点值数组乘上 B 就可以得到答案了。

结论就是 $B_{i,j} = \frac{\omega^{-ij}}{N}$ 。证明很简单, 直接把 AB 乘起来, 发现只有 $i = j$ 时结果为 1, 其余为 0。

于是考虑快速计算 CB 的结果, 其实和 DFT 是一样的, 不妨把这个 $\frac{1}{n}$ 放到最后再除, 其他无非就是把单位根幂次换成了负的。

于是也做完了。恭喜你已经学会了朴素的 FFT 算法。



FFT 快速傅里叶变换算法

偷了一个别人写的代码：



FFT 快速傅里叶变换算法

偷了一个别人写的代码：

```
I^^I^^Iconst double Pi = acos(-1.0);
I^^I^^Ivoid FFT(int limit, complex<double> *a, int type) {
I^^I^^I^^Iif (limit == 1) return;
I^^I^^I^^Icomplex<double> a1[limit >> 1], a2[limit >> 1];
I^^I^^I^^Ifor (int i = 0; i < limit; i += 2) {
I^^I^^I^^I^^Ia1[i >> 1] = a[i];
I^^I^^I^^I^^Ia2[i >> 1] = a[i + 1];
I^^I^^I^^I}
I^^I^^I^^IFFT(limit >> 1, a1, type);
I^^I^^I^^IFFT(limit >> 1, a2, type);
I^^I^^I^^Icomplex<double> Wn(cos(2.0 * Pi / limit), type * sin(2.0 * Pi / limit)), w(1, 0);
I^^I^^I^^Ifor (int i = 0; i < (limit >> 1); i++, w = w * Wn) {
I^^I^^I^^I^^Ia[i] = a1[i] + w * a2[i];
I^^I^^I^^I^^Ia[i + (limit >> 1)] = a1[i] - w * a2[i];
I^^I^^I^^I}
I^^I^^I}
I^^I}
```



FFT 快速傅里叶变换算法

上述代码是递归实现的，效率不优。



FFT 快速傅里叶变换算法

上述代码是递归实现的，效率不优。

容易发现，递归过程其实就是奇偶分类，如果我们把每一个数字的二进制位给翻转掉，然后再递归，其实就是从低位到高位向上合并的过程。

FFT 快速傅里叶变换算法

上述代码是递归实现的，效率不优。

容易发现，递归过程其实就是奇偶分类，如果我们把每一个数字的二进制位给翻转掉，然后再递归，其实就是从低位到高位向上合并的过程。

于是我们可以写出递推形式的代码：

```
void fast_fast_tle(complex *A,int type){
    for(int i=0;i<limit;i++)
        if(i<r[i]) swap(A[i],A[r[i]]);
    for(int mid=1;mid<limit;mid<=<=1){
        complex Wn( cos(Pi/mid) , type*sin(Pi/mid) );
        for(int R=mid<<1,j=0;j<limit;j+=R){
            complex w(1,0);
            for(int k=0;k<mid;k++,w=w*Wn){
                complex x=A[j+k],y=w*A[j+mid+k];
                A[j+k]=x+y,A[j+mid+k]=x-y;
            }
        }
    }
}
```



NTT 快速数论变化算法

通过 FFT 可以看出，这个算法是有精度误差的，并且浮点数用起来一直都不是很舒服。如果在模意义下有和单位根性质完全相同的东西，那么我们就可以去带入这些点的点值了。



NTT 快速数论变化算法

通过 FFT 可以看出，这个算法是有精度误差的，并且浮点数用起来一直都不是很舒服。如果在模意义下有和单位根性质完全相同的东西，那么我们就可以去带入这些点的点值了。



NTT 快速数论变化算法

通过 FFT 可以看出，这个算法是有精度误差的，并且浮点数用起来一直都不是很舒服。如果在模意义下有和单位根性质完全相同的东西，那么我们就可以去带入这些点的点值了。
所以科普一个东西，在模质数的意义下，存在一个东西叫做原根 g 。



NTT 快速数论变化算法

通过 FFT 可以看出，这个算法是有精度误差的，并且浮点数用起来一直都不是很舒服。如果在模意义下有和单位根性质完全相同的东西，那么我们就可以去带入这些点的点值了。

所以科普一个东西，在模质数的意义下，存在一个东西叫做原根 g 。

他满足一个性质： $g^{p-1} \equiv 1 \pmod{p}$ ，并且第一个让 $g^i \equiv 1$ 的就是 $p-1$ 。那么可以发现， g^i 可以取遍 $1 \sim p-1$ 内的数。



NTT 快速数论变化算法

通过 FFT 可以看出，这个算法是有精度误差的，并且浮点数用起来一直都不是很舒服。如果在模意义下有和单位根性质完全相同的东西，那么我们就可以去带入这些点的点值了。

所以科普一个东西，在模质数的意义下，存在一个东西叫做原根 g 。

他满足一个性质： $g^{p-1} \equiv 1 \pmod{p}$ ，并且第一个让 $g^i \equiv 1$ 的就是 $p-1$ 。那么可以发现， g^i 可以取遍 $1 \sim p-1$ 内的数。

而原根不止一个，但是质数都存在原根。



NTT 快速数论变化算法

通过 FFT 可以看出，这个算法是有精度误差的，并且浮点数用起来一直都不是很舒服。如果在模意义下有和单位根性质完全相同的东西，那么我们就可以去带入这些点的点值了。

所以科普一个东西，在模质数的意义下，存在一个东西叫做原根 g 。

他满足一个性质： $g^{p-1} \equiv 1 \pmod{p}$ ，并且第一个让 $g^i \equiv 1$ 的就是 $p-1$ 。那么可以发现， g^i 可以取遍 $1 \sim p-1$ 内的数。

而原根不止一个，但是质数都存在原根。

所以 g 的幂可以构成一个在 $1 \sim p-1$ 的环，于是 $g^{\frac{p-1}{2^N}} = \omega_N^1$ 就是我们想要的。把上述做法的单位根换成它就好了。不过值得注意的是，此处必须满足 $2^N | p-1$ ，这样的质数不一定多，最经典的就是 998244353。所以只有模数是这样的 $p-1$ 有大量二的因子时，可以用这个算法。



NTT 快速数论变化算法

通过 FFT 可以看出，这个算法是有精度误差的，并且浮点数用起来一直都不是很舒服。如果在模意义下有和单位根性质完全相同的东西，那么我们就可以去带入这些点的点值了。

所以科普一个东西，在模质数的意义下，存在一个东西叫做原根 g 。

他满足一个性质： $g^{p-1} \equiv 1 \pmod{p}$ ，并且第一个让 $g^i \equiv 1$ 的就是 $p-1$ 。那么可以发现， g^i 可以取遍 $1 \sim p-1$ 内的数。

而原根不止一个，但是质数都存在原根。

所以 g 的幂可以构成一个在 $1 \sim p-1$ 的环，于是 $g^{\frac{p-1}{2^N}} = \omega_N^1$ 就是我们想要的。把上述做法的单位根换成它就好了。不过值得注意的是，此处必须满足 $2^N | p-1$ ，这样的质数不一定多，最经典的就是 998244353。所以只有模数是这样的 $p-1$ 有大量二的因子时，可以用这个算法。

代码很简单就不给了。



例题

P1919



例题

P1919, P4245



例题

P1919, P4245, P12735



例题

P1919, P4245, P12735, P4173



例题

P1919, P4245, P12735, P4173, P5641



例题

P1919, P4245, P12735, P4173, P5641, CF1975G



例题

P1919, P4245, P12735, P4173, P5641, CF1975G, P12735



让我们再次回顾 NTT 的原理

湖南省队集训 2025 D2T2:



让我们再次回顾 NTT 的原理

湖南省队集训 2025 D2T2:

给出 n , 对于所有的 $1 \leq m \leq n$, 计算 $\sum_{i=1}^n \binom{n-i}{i} \binom{m-i}{i}$ 。



让我们再次回顾 NTT 的原理

湖南省队集训 2025 D2T2:

给出 n , 对于所有的 $1 \leq m \leq n$, 计算 $\sum_{i=1}^n \binom{n-i}{i} \binom{m-i}{i}$ 。

$1 \leq n \leq 10^6$ 。



多项式求逆

给出多项式 $A(x)$, 计算 $B(x)$, 满足 $A(x)B(x) = 1 \bmod x^{n+1}$ 。



多项式求逆

给出多项式 $A(x)$ ，计算 $B(x)$ ，满足 $A(x)B(x) = 1 \bmod x^{n+1}$ 。
我们可以暴力的解方程，但那样是平方复杂度的，所以请出牛顿迭代。



多项式求逆

给出多项式 $A(x)$, 计算 $B(x)$, 满足 $A(x)B(x) = 1 \bmod x^{n+1}$ 。
我们可以暴力的解方程, 但那样是平方复杂度的, 所以请出牛顿迭代。
假如我想解一个多项式方程 $G(F(x)) = 0$, 求出 $F \bmod x^n$ 的结果。

多项式求逆

给出多项式 $A(x)$, 计算 $B(x)$, 满足 $A(x)B(x) = 1 \bmod x^{n+1}$ 。

我们可以暴力的解方程, 但那样是平方复杂度的, 所以请出牛顿迭代。

假如我想解一个多项式方程 $G(F(x)) = 0$, 求出 $F \bmod x^n$ 的结果。

我们分治, 先求出 $F(x) \bmod x^{n/2}$ (上取整)。考虑如何扩展到 $\bmod x^n$ 下, 可以把 $G(F(z))$ 在 $F_0(z)$ 这里进行泰勒展开

$$\begin{aligned}
 G(F(z)) &= G(F_0(z)) \\
 &+ \frac{G'(F_0(z))}{1!} (F(z) - F_0(z)) \\
 &+ \frac{G''(F_0(z))}{2!} (F(z) - F_0(z))^2 \\
 &+ \dots
 \end{aligned}$$

多项式求逆

给出多项式 $A(x)$, 计算 $B(x)$, 满足 $A(x)B(x) = 1 \bmod x^{n+1}$ 。

我们可以暴力的解方程, 但那样是平方复杂度的, 所以请出牛顿迭代。

假如我想解一个多项式方程 $G(F(x)) = 0$, 求出 $F \bmod x^n$ 的结果。

我们分治, 先求出 $F(x) \bmod x^{n/2}$ (上取整)。考虑如何扩展到 $\bmod x^n$ 下, 可以把 $G(F(z))$ 在 $F_0(z)$ 这里进行泰勒展开

$$\begin{aligned}
 G(F(z)) &= G(F_0(z)) \\
 &+ \frac{G'(F_0(z))}{1!} (F(z) - F_0(z)) \\
 &+ \frac{G''(F_0(z))}{2!} (F(z) - F_0(z))^2 \\
 &+ \dots
 \end{aligned}$$

因为 $F(z)$ 和 $F_0(z)$ 的最后 $\lceil \frac{n}{2} \rceil$ 项相同, 所以 $(F(z) - F_0(z))^2$ 的最低的非 0 项次数大于 $2 \lceil \frac{n}{2} \rceil$, 所以可以得到

多项式求逆

$$G(F(z)) \equiv G(F_0(z)) + G'(F_0(z))(F(z) - F_0(z)) \pmod{z^n}$$



多项式求逆

$$G(F(z)) \equiv G(F_0(z)) + G'(F_0(z))(F(z) - F_0(z)) \pmod{z^n}$$

然后因为 $G(F(z)) \equiv 0 \pmod{z^n}$, 可以得到

$$F(z) \equiv F_0(z) - \frac{G(F_0(z))}{G'(F_0(z))} \pmod{z^n}$$



多项式求逆

$$G(F(z)) \equiv G(F_0(z)) + G'(F_0(z))(F(z) - F_0(z)) \pmod{z^n}$$

然后因为 $G(F(z)) \equiv 0 \pmod{z^n}$, 可以得到

$$F(z) \equiv F_0(z) - \frac{G(F_0(z))}{G'(F_0(z))} \pmod{z^n}$$

然后好像就完了? 现在来看看它能干什么用好了。



多项式求逆

$$G(F(z)) \equiv G(F_0(z)) + G'(F_0(z))(F(z) - F_0(z)) \pmod{z^n}$$

然后因为 $G(F(z)) \equiv 0 \pmod{z^n}$, 可以得到

$$F(z) \equiv F_0(z) - \frac{G(F_0(z))}{G'(F_0(z))} \pmod{z^n}$$

然后好像就完了? 现在来看看它能干什么用好了。

令 $G(F(x)) = \frac{1}{F(x)} - A(x)$ 。



多项式求逆

$$G(F(z)) \equiv G(F_0(z)) + G'(F_0(z))(F(z) - F_0(z)) \pmod{z^n}$$

然后因为 $G(F(z)) \equiv 0 \pmod{z^n}$, 可以得到

$$F(z) \equiv F_0(z) - \frac{G(F_0(z))}{G'(F_0(z))} \pmod{z^n}$$

然后好像就完了? 现在来看看它能干什么用好了。

令 $G(F(x)) = \frac{1}{F(x)} - A(x)$ 。

带入解得: $F_1(x) = F_0(x)(2 - A(x)F_0(x))$ 。



多项式求逆

$$G(F(z)) \equiv G(F_0(z)) + G'(F_0(z))(F(z) - F_0(z)) \pmod{z^n}$$

然后因为 $G(F(z)) \equiv 0 \pmod{z^n}$, 可以得到

$$F(z) \equiv F_0(z) - \frac{G(F_0(z))}{G'(F_0(z))} \pmod{z^n}$$

然后好像就完了? 现在来看看它能干什么用好了。

令 $G(F(x)) = \frac{1}{F(x)} - A(x)$ 。

带入解得: $F_1(x) = F_0(x)(2 - A(x)F_0(x))$ 。

递归即可。



例题

P4841, P4723, 还有一个例题在后面。



目录

1 多项式算法

2 多项式插值

3 生成函数

有一个 n 次多项式 $f(x)$ (未知)。已知其 $n+1$ 个点值 (x_i, y_i) ，按理来说我已经可以确定 $f(x)$ ，现在我想求出来 $f(x_0)$ 的值怎么办。



拉格朗日插值

算法的核心思路是这样的，既然我们知道满足这个条件的 n 次多项式唯一，考虑使用某种方法构造出来其中任何一种可以满足条件的结果即可，我们这样做：



拉格朗日插值

算法的核心思路是这样的，既然我们知道满足这个条件的 n 次多项式唯一，考虑使用某种方法构造出来其中任何一种可以满足条件的结果即可，我们这样做：

设 $f_i(x)$ 满足 $f_i(x_j) = 0, j \neq i, f_i(x_i) = 1$ 。



拉格朗日插值

算法的核心思路是这样的，既然我们知道满足这个条件的 n 次多项式唯一，考虑使用某种方法构造出来其中任何一种可以满足条件的结果即可，我们这样做：

设 $f_i(x)$ 满足 $f_i(x_j) = 0, j \neq i, f_i(x_i) = 1$ 。

于是 $f(x) = \sum_{i=1}^n f_i(x)y_i$ 。



拉格朗日插值

算法的核心思路是这样的，既然我们知道满足这个条件的 n 次多项式唯一，考虑使用某种方法构造出来其中任何一种可以满足条件的结果即可，我们这样做：

设 $f_i(x)$ 满足 $f_i(x_j) = 0, j \neq i, f_i(x_i) = 1$ 。

于是 $f(x) = \sum_{i=1}^n f_i(x)y_i$ 。

考虑求出 $f_i(x)$ ，容易得到这样的构造： $f_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$ 。



拉格朗日插值

算法的核心思路是这样的，既然我们知道满足这个条件的 n 次多项式唯一，考虑使用某种方法构造出来其中任何一种可以满足条件的结果即可，我们这样做：

设 $f_i(x)$ 满足 $f_i(x_j) = 0, j \neq i, f_i(x_i) = 1$ 。

于是 $f(x) = \sum_{i=1}^n f_i(x) y_i$ 。

考虑求出 $f_i(x)$ ，容易得到这样的一个构造： $f_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$ 。

于是得到拉格朗日插值公式 $f(x) = \sum_{i=1}^n y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$ 。



拉格朗日插值

算法的核心思路是这样的，既然我们知道满足这个条件的 n 次多项式唯一，考虑使用某种方法构造出来其中任何一种可以满足条件的结果即可，我们这样做：

设 $f_i(x)$ 满足 $f_i(x_j) = 0, j \neq i, f_i(x_i) = 1$ 。

于是 $f(x) = \sum_{i=1}^n f_i(x) y_i$ 。

考虑求出 $f_i(x)$ ，容易得到这样的构造： $f_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$ 。

于是得到拉格朗日插值公式 $f(x) = \sum_{i=1}^n y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$ 。

直接带入计算得到 $f(x_0)$ 即可。



拉格朗日插值

算法的核心思路是这样的，既然我们知道满足这个条件的 n 次多项式唯一，考虑使用某种方法构造出来其中任何一种可以满足条件的结果即可，我们这样做：

设 $f_i(x)$ 满足 $f_i(x_j) = 0, j \neq i, f_i(x_i) = 1$ 。

于是 $f(x) = \sum_{i=1}^n f_i(x) y_i$ 。

考虑求出 $f_i(x)$ ，容易得到这样的一个构造： $f_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$ 。

于是得到拉格朗日插值公式 $f(x) = \sum_{i=1}^n y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$ 。

直接带入计算得到 $f(x_0)$ 即可。

时间复杂度 $O(n^2)$ 。



拉格朗日插值

如果点值比较特殊呢？假如我们有 $x_i = i$ ，即已知点的横坐标是连续的，能不能做的优？

拉格朗日插值

如果点值比较特殊呢？假如我们有 $x_i = i$ ，即已知点的横坐标是连续的，能不能做的优？
显然可以，直接优化后面这个求积即可，维护前缀积与后缀积，分别算分子分母即可快速计算。



例题

一般的拉插题目都是让你先写出一个式子，或者 dp。你要去猜测它是否为多项式（这个纯依靠感觉），算出部分点值后，再使用拉插优化。

例题

一般的拉插题目都是让你先写出一个式子，或者 dp。你要去猜测它是否为多项式（这个纯依靠感觉），算出部分点值后，再使用拉插优化。

不过也是有迹可循的，如果一个 n 次多项式 $f(x)$ 乘一个 $O(x)$ 的数字，次数加一，做一次系数上的前缀和，次数在加一，我们可以用这两个法则来辅助我们的思考。



例题

一般的拉插题目都是让你先写出一个式子，或者 dp。你要去猜测它是否为多项式（这个纯依靠感觉），算出部分点值后，再使用拉插优化。

不过也是有迹可循的，如果一个 n 次多项式 $f(x)$ 乘一个 $O(x)$ 的数字，次数加一，做一次系数上的前缀和，次数在加一，我们可以用这两个法则来辅助我们的思考。

P4463



例题

一般的拉插题目都是让你先写出一个式子，或者 dp。你要去猜测它是否为多项式（这个纯依靠感觉），算出部分点值后，再使用拉插优化。

不过也是有迹可循的，如果一个 n 次多项式 $f(x)$ 乘一个 $O(x)$ 的数字，次数加一，做一次系数上的前缀和，次数在加一，我们可以用这两个法则来辅助我们的思考。

P4463, P5667



例题

一般的拉插题目都是让你先写出一个式子，或者 dp。你要去猜测它是否为多项式（这个纯依靠感觉），算出部分点值后，再使用拉插优化。

不过也是有迹可循的，如果一个 n 次多项式 $f(x)$ 乘一个 $O(x)$ 的数字，次数加一，做一次系数上的前缀和，次数在加一，我们可以用这两个法则来辅助我们的思考。

P4463, P5667, CF622F

例题

一般的拉插题目都是让你先写出一个式子，或者 dp。你要去猜测它是否为多项式（这个纯依靠感觉），算出部分点值后，再使用拉插优化。

不过也是有迹可循的，如果一个 n 次多项式 $f(x)$ 乘一个 $O(x)$ 的数字，次数加一，做一次系数上的前缀和，次数在加一，我们可以用这两个法则来辅助我们的思考。

P4463, P5667, CF622F, P8290

例题

一般的拉插题目都是让你先写出一个式子，或者 dp。你要去猜测它是否为多项式（这个纯依靠感觉），算出部分点值后，再使用拉插优化。

不过也是有迹可循的，如果一个 n 次多项式 $f(x)$ 乘一个 $O(x)$ 的数字，次数加一，做一次系数上的前缀和，次数在加一，我们可以用这两个法则来辅助我们的思考。

P4463, P5667, CF622F, P8290, P13042



系数插系数

当然偶尔还会遇见要重新复原系数的问题，这也正常，怎么办呢。



系数插系数

当然偶尔还会遇见要重新复原系数的问题，这也正常，怎么办呢。
直接对拉插的式子做退背包即可，也就是每次除一个一次函数，再乘一个回来。



系数插系数

当然偶尔还会遇见要重新复原系数的问题，这也正常，怎么办呢。
直接对拉插的式子做退背包即可，也就是每次除一个一次函数，再乘一个回来。
这个东西应用，我忘记有什么题目了（悲。



系数插系数

当然偶尔还会遇见要重新复原系数的问题，这也正常，怎么办呢。
直接对拉插的式子做退背包即可，也就是每次除一个一次函数，再乘一个回来。
这个东西应用，我忘记有什么题目了（悲。
牛顿插值不用学



目录

1 多项式算法

2 多项式插值

3 生成函数



生成函数

用函数来刻画序列，或者组合对象的方式。



生成函数

用函数来刻画序列，或者组合对象的方式。
一般有两种生成函数：



生成函数

用函数来刻画序列，或者组合对象的方式。

一般有两种生成函数：

- 对于数列 a ，定义其生成函数 $F(x) = \sum_i a_i x^i$



生成函数

用函数来刻画序列，或者组合对象的方式。

一般有两种生成函数：

- 对于数列 a ，定义其生成函数 $F(x) = \sum_i a_i x^i$
- 对于一个组合对象，设 S 为我们需要计数的目标集合，设 $F(x) = \sum_{a \in S} x^{|a|}$ 。

用函数来刻画序列，或者组合对象的方式。

一般有两种生成函数：

- 对于数列 a , 定义其生成函数 $F(x) = \sum_i a_i x^i$
- 对于一个组合对象, 设 S 为我们需要计数的目标集合, 设 $F(x) = \sum_{a \in S} x^{|a|}$.

二者都有各自的作用，而在一般的计数问题中，我认为直接用方法二求出对象的生成函数会更直接，更本质。



生成函数

一个初学者容易考虑的问题是，一个函数，为什么只关心系数，而不在意它在某些位置的值，这样有什么意义？



生成函数

一个初学者容易考虑的问题是，一个函数，为什么只关心系数，而不在意它在某些位置的值，这样有什么意义？

我个人的解答是这样的：生成函数就像矩阵快速幂一样，是用来刻画操作的。同时把它写成函数，只是为了寻求代数上的等价变形，方便我们操作并且简化问题。



生成函数

一个初学者容易考虑的问题是，一个函数，为什么只关心系数，而不在意它在某些位置的值，这样有什么意义？

我个人的解答是这样的：生成函数就像矩阵快速幂一样，是用来刻画操作的。同时把它写成函数，只是为了寻求代数上的等价变形，方便我们操作并且简化问题。

比如说计数背包的本质就是生成函数的乘法，也就是把两个组合对象卷积在了一起。

生成函数

一个初学者容易考虑的问题是，一个函数，为什么只关心系数，而不在意它在某些位置的值，这样有什么意义？

我个人的解答是这样的：生成函数就像矩阵快速幂一样，是用来刻画操作的。同时把它写成函数，只是为了寻求代数上的等价变形，方便我们操作并且简化问题。

比如说计数背包的本质就是生成函数的乘法，也就是把两个组合对象卷积在了一起。

所以生成函数的点值一般不重要，他只是“形式幂级数”，只有级数的形式罢了，所以我们对生成函数操作无需在意其敛散性，只需要你想这么做就好了（。



生成函数

一个初学者容易考虑的问题是，一个函数，为什么只关心系数，而不在意它在某些位置的值，这样有什么意义？

我个人的解答是这样的：生成函数就像矩阵快速幂一样，是用来刻画操作的。同时把它写成函数，只是为了寻求代数上的等价变形，方便我们操作并且简化问题。

比如说计数背包的本质就是生成函数的乘法，也就是把两个组合对象卷积在了一起。

所以生成函数的点值一般不重要，他只是“形式幂级数”，只有级数的形式罢了，所以我们对生成函数操作无需在意其敛散性，只需要你想这么做就好了（。

经典的例子是，有生成函数 $A(x)$ ，我要计算 $1 + A + A^2 + A^3 + A^4 + \dots$ ，其实就是 $\frac{1}{1 - A(x)}$ ，因为我默认收敛。



生成函数

因此有很多经典的生成函数，我们一个个看：



生成函数

因此有很多经典的生成函数，我们一个个看：
求序列 $1, 1, 1, \dots$ 的生成函数。



生成函数

因此有很多经典的生成函数，我们一个个看：

求序列 $1, 1, 1, \dots$ 的生成函数。

求序列 $1, 2, 4, 8, \dots$ 的生成函数。



生成函数

因此有很多经典的生成函数，我们一个个看：

求序列 $1, 1, 1, \dots$ 的生成函数。

求序列 $1, 2, 4, 8, \dots$ 的生成函数。

求序列 $1, 0, 1, 0, \dots$ 的生成函数。



生成函数

因此有很多经典的生成函数，我们一个个看：

求序列 $1, 1, 1, \dots$ 的生成函数。

求序列 $1, 2, 4, 8, \dots$ 的生成函数。

求序列 $1, 0, 1, 0, \dots$ 的生成函数。

求序列 $1, 2, 3, \dots$ 的生成函数。



生成函数

因此有很多经典的生成函数，我们一个个看：

求序列 $1, 1, 1, \dots$ 的生成函数。

求序列 $1, 2, 4, 8, \dots$ 的生成函数。

求序列 $1, 0, 1, 0, \dots$ 的生成函数。

求序列 $1, 2, 3, \dots$ 的生成函数。

求序列 $a_i = \binom{n}{i}$ 的生成函数。



生成函数

因此有很多经典的生成函数，我们一个个看：

求序列 $1, 1, 1, \dots$ 的生成函数。

求序列 $1, 2, 4, 8, \dots$ 的生成函数。

求序列 $1, 0, 1, 0, \dots$ 的生成函数。

求序列 $1, 2, 3, \dots$ 的生成函数。

求序列 $a_i = \binom{n}{i}$ 的生成函数。

求序列 $a_n = \binom{n+m}{n}$ 的生成函数。



生成函数

因此有很多经典的生成函数，我们一个个看：

求序列 $1, 1, 1, \dots$ 的生成函数。

求序列 $1, 2, 4, 8, \dots$ 的生成函数。

求序列 $1, 0, 1, 0, \dots$ 的生成函数。

求序列 $1, 2, 3, \dots$ 的生成函数。

求序列 $a_i = \binom{n}{i}$ 的生成函数。

求序列 $a_n = \binom{n+m}{n}$ 的生成函数。

求斐波那契数列的生成函数。



生成函数

因此有很多经典的生成函数，我们一个个看：

求序列 $1, 1, 1, \dots$ 的生成函数。

求序列 $1, 2, 4, 8, \dots$ 的生成函数。

求序列 $1, 0, 1, 0, \dots$ 的生成函数。

求序列 $1, 2, 3, \dots$ 的生成函数。

求序列 $a_i = \binom{n}{i}$ 的生成函数。

求序列 $a_n = \binom{n+m}{n}$ 的生成函数。

求斐波那契数列的生成函数。

求卡特兰数数列的生成函数 $C(x)$ 。



生成函数

因此有很多经典的生成函数，我们一个个看：

求序列 $1, 1, 1, \dots$ 的生成函数。

求序列 $1, 2, 4, 8, \dots$ 的生成函数。

求序列 $1, 0, 1, 0, \dots$ 的生成函数。

求序列 $1, 2, 3, \dots$ 的生成函数。

求序列 $a_i = \binom{n}{i}$ 的生成函数。

求序列 $a_n = \binom{n+m}{n}$ 的生成函数。

求斐波那契数列的生成函数。

求卡特兰数数列的生成函数 $C(x)$ 。

求 $[x^n] C^k(x)$ 。



生成函数

因此有很多经典的生成函数，我们一个个看：

求序列 $1, 1, 1, \dots$ 的生成函数。

求序列 $1, 2, 4, 8, \dots$ 的生成函数。

求序列 $1, 0, 1, 0, \dots$ 的生成函数。

求序列 $1, 2, 3, \dots$ 的生成函数。

求序列 $a_i = \binom{n}{i}$ 的生成函数。

求序列 $a_n = \binom{n+m}{n}$ 的生成函数。

求斐波那契数列的生成函数。

求卡特兰数数列的生成函数 $C(x)$ 。

求 $[x^n] C^k(x)$ 。

反射容斥的例题：P14973, P3266



生成函数

让我们直观地感受直接将组合对象视作为生成函数，考虑这样的问题：



生成函数

让我们直观地感受直接将组合对象视作为生成函数，考虑这样的问题：
在许多不同种类的食物中选出 n 个，每种食物的限制如下：



生成函数

让我们直观地感受直接将组合对象视为生成函数，考虑这样的问题：
在许多不同种类的食物中选出 n 个，每种食物的限制如下：

- 1 承德汉堡：偶数个
- 2 可乐：0 个或 1 个
- 3 鸡腿：0 个，1 个或 2 个
- 4 蜜桃多：奇数个
- 5 鸡块：4 的倍数个
- 6 包子：0 个，1 个，2 个或 3 个
- 7 土豆片炒肉：不超过一个
- 8 面包：3 的倍数个

每种食物都是以「个」为单位，只要总数加起来是 n 就算一种方案。对于给出的 n 你需要计算出方案数，对 10007 取模。



生成函数

让我们直观地感受直接将组合对象视为生成函数，考虑这样的问题：
在许多不同种类的食物中选出 n 个，每种食物的限制如下：

- 1 承德汉堡：偶数个
- 2 可乐：0 个或 1 个
- 3 鸡腿：0 个，1 个或 2 个
- 4 蜜桃多：奇数个
- 5 鸡块：4 的倍数个
- 6 包子：0 个，1 个，2 个或 3 个
- 7 土豆片炒肉：不超过一个
- 8 面包：3 的倍数个

每种食物都是以「个」为单位，只要总数加起来是 n 就算一种方案。对于给出的 n 你需要计算出方案数，对 10007 取模。

求出各个实物的生成函数，再用背包合并，也就是乘起来就好了。



生成函数

再来一个例子：



生成函数

再来一个例子：

求不定方程 $\sum_{i=1}^m x_i = n$ 的方案数，其中 x_i 是自然数。



生成函数

把所有需要维护的元素看成生成函数中的“元”。



生成函数

把所有需要维护的元素看成生成函数中的“元”。
计算满足以下条件的序列数量：

生成函数

把所有需要维护的元素看成生成函数中的“元”。

计算满足以下条件的序列数量：

- 长度为 n ，值域是 $[1, a + b]$ 中的整数。



生成函数

把所有需要维护的元素看成生成函数中的“元”。

计算满足以下条件的序列数量：

- 长度为 n ，值域是 $[1, a + b]$ 中的整数。
- 所有 $\leq a$ 的数字，出现次数均为奇数。



生成函数

把所有需要维护的元素看成生成函数中的“元”。

计算满足以下条件的序列数量：

- 长度为 n ，值域是 $[1, a + b]$ 中的整数。
- 所有 $\leq a$ 的数字，出现次数均为奇数。
- 所有 $> a$ 的数字，出现次数均为偶数。



生成函数

把所有需要维护的元素看成生成函数中的“元”。

计算满足以下条件的序列数量：

- 长度为 n ，值域是 $[1, a + b]$ 中的整数。
- 所有 $\leq a$ 的数字，出现次数均为奇数。
- 所有 $> a$ 的数字，出现次数均为偶数。

答案对某个质数 p 取模。



生成函数

把所有需要维护的元素看成生成函数中的“元”。

计算满足以下条件的序列数量：

- 长度为 n ，值域是 $[1, a + b]$ 中的整数。
- 所有 $\leq a$ 的数字，出现次数均为奇数。
- 所有 $> a$ 的数字，出现次数均为偶数。

答案对某个质数 p 取模。



生成函数

把所有需要维护的元素看成生成函数中的“元”。

计算满足以下条件的序列数量：

- 长度为 n ，值域是 $[1, a + b]$ 中的整数。
- 所有 $\leq a$ 的数字，出现次数均为奇数。
- 所有 $> a$ 的数字，出现次数均为偶数。

答案对某个质数 p 取模。

$1 \leq n \leq 10^9, 1 \leq a, b \leq 1000$ 。



例题

P6078



例题

P6078,P4451



例题

P6078,P4451,P3978



将组合对象看做生成函数

湖南省队集训 2025 D1T1:



将组合对象看做生成函数

湖南省队集训 2025 D1T1:

求满足以下条件的序列个数:

- 长度为 n , 值域是 $[1, m]$ 的整数。
- $[1, m]$ 内每一个数都出现至少一次。
- 相邻的两个数不相同。
- 所有长度为 m 的区间, 至少有两个位置上面的数字相同。

答案对 998244353 取模。

$1 \leq m \leq 10^5, 1 \leq n \leq 10^9$ 。